

Project Report: Predictive Modeling for Income Classification Using the Census Income Dataset

Name: Bhavika Sood

Person Number: 50604788

Project Title: Income Classification Using the Census Income Dataset

Objective: The aim of this project is to classify whether an individual earns more than USD 50,000/year based on census data, using preprocessing techniques and predictive modeling with a classifier to achieve at least 85% accuracy.

Instructor's Name: Dr. Asif Imran

Course Number and Name: CSE 574: Introduction to Machine Learning

INTRODUCTION

This project aims to classify individuals' income levels using the Adult Census Income dataset, focusing on whether they earn more than USD 50,000 per year. The dataset includes key demographic features like age, education, occupation, gender, and marital status, which are crucial in influencing earning potential. By preprocessing the data, exploring these features, and applying a **Gradient Boosting Classifier**, the project seeks to uncover patterns that impact income. This analysis demonstrates how machine learning can be used to predict income levels, offering valuable insights for socio-economic research and policy-making.

PROJECT TASKS

1. Load the dataset

```
import pandas as pd

data = '/Users/bhavikasood/Downloads/Project_1_python/dataset_adult/adult.data'
test = '/Users/bhavikasood/Downloads/Project_1_python/dataset_adult/adult.test'

columns = ['age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status', 'occupation',
           'relationship', 'race', 'sex', 'capital_gain', 'capital_loss', 'hours_per_week', 'native_country', 'income']

adult_data = pd.read_csv(data, names=columns, na_values='?', skipinitialspace=True)
adult_test = pd.read_csv(test, names=columns, na_values='?', skipinitialspace=True, skiprows=1)

print("\nAdult Training Data:\n")
display(adult_data.head())

print("\nAdult Test Data:\n")
display(adult_test.head())
```

The code snippet above imports the necessary data from the Adult Census Income dataset files (**adult.data** and **adult.test**) into **pandas DataFrames** for analysis. The paths to these files are specified as strings, pointing to their locations on the local machine.

Since the dataset does not include headers, a list of column names is defined to match the attributes present in the data, such as age, workclass, education, and income. These names are then passed to the **pd.read_csv** function to load the data correctly.

The **pd.read_csv** function is used with several parameters:

- **names=columns** assign the predefined column names to the DataFrame.
- **na_values='?'** treats any occurrence of ? in the dataset as a missing value (NaN).
- **skipinitialspace=True** helps clean up any extra spaces following delimiters.
- **skiprows=1** is used when reading the test dataset to ignore an unwanted header line.

Finally, the code displays the first few rows of both the training and test datasets using **display (adult_data.head())** and **display (adult_test.head())**.

Adult Training Data:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

Adult Test Data:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
0	25	Private	226802	11th	7	Never-married	Machine-op-inspct	Own-child	Black	Male	0	0	40	United-States	<=50K.
1	38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing	Husband	White	Male	0	0	50	United-States	<=50K.
2	28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv	Husband	White	Male	0	0	40	United-States	>50K.
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	Husband	Black	Male	7688	0	40	United-States	>50K.
4	18	NaN	103497	Some-college	10	Never-married	NaN	Own-child	White	Female	0	0	30	United-States	<=50K.

2. Output the structure of the dataset

```
adult_datainfo = adult_data.info()
print('\n')
adult_testinfo = adult_test.info()
```

This code snippet provides a detailed summary of the structure of both the training (**adult_data**) and test (**adult_test**) datasets. The **.info()** method is called on each DataFrame, which outputs the following information:

- **Column Names:** Lists each column in the dataset.
- **Data Types:** Displays the data type of each column (e.g., int64, object).

- **Non-Null Counts:** Shows the number of non-missing entries for each column, which is useful for identifying columns with missing values.
- **Memory Usage:** Indicates the memory usage of the DataFrame, giving an idea of the dataset's size in memory.

This output helps identify the columns that need cleaning (those with missing values) and confirms the overall structure of the datasets.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   32561 non-null  int64
1   workclass             30725 non-null  object
2   fnlwgt                32561 non-null  int64
3   education             32561 non-null  object
4   education_num         32561 non-null  int64
5   marital_status        32561 non-null  object
6   occupation            30718 non-null  object
7   relationship          32561 non-null  object
8   race                  32561 non-null  object
9   sex                   32561 non-null  object
10  capital_gain          32561 non-null  int64
11  capital_loss          32561 non-null  int64
12  hours_per_week        32561 non-null  int64
13  native_country        31978 non-null  object
14  income                32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16281 entries, 0 to 16280
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   16281 non-null  int64
1   workclass             15318 non-null  object
2   fnlwgt                16281 non-null  int64
3   education             16281 non-null  object
4   education_num         16281 non-null  int64
5   marital_status        16281 non-null  object
6   occupation            15315 non-null  object
7   relationship          16281 non-null  object
8   race                  16281 non-null  object
9   sex                   16281 non-null  object
10  capital_gain          16281 non-null  int64
11  capital_loss          16281 non-null  int64
12  hours_per_week        16281 non-null  int64
13  native_country        16007 non-null  object
14  income                16281 non-null  object
dtypes: int64(6), object(9)
memory usage: 1.9+ MB
```

3. Clean the dataset

Cleaning and handling the missing values

```
print("Missing Values in the Training Data Before Cleaning")
print(adult_data.isnull().sum())

print("\nMissing Values in the Test Data Before Cleaning")
print(adult_test.isnull().sum())

clean_adult_data = adult_data.dropna()
clean_adult_test = adult_test.dropna()

clean_adult_test['income'] = clean_adult_test['income'].str.replace('.', '')

print("\nMissing Values in the Training Data After Cleaning")
print(clean_adult_data.isnull().sum())

print("\nMissing Values in the Test Data After Cleaning")
print(clean_adult_test.isnull().sum())

print("\nAdult Training Data (after cleaning):\n")
display(clean_adult_data.head())
print("\nAdult Test Data (after cleaning):\n")
display(clean_adult_test.head())
```

This code snippet focuses on identifying and handling missing values in the dataset. It also cleans up specific formatting issues. Each function used does the following:

- **isnull().sum()**: This function checks for missing values in both the training (`adult_data`) and test (`adult_test`) datasets. It returns the count of missing values for each column, helping to identify which columns require cleaning.
- **dropna()**: This function is applied to remove all rows containing missing values in both datasets. By dropping these rows, the cleaned datasets (`clean_adult_data` and `clean_adult_test`) are left with complete entries, free of any null values.
- **str.replace('.', '')**: This function is used to fix formatting in the income column of the test dataset. The income column contains trailing periods (e.g., `>50K.` and `<=50K.`). By using `str.replace('.', '')`, the code removes these periods, standardizing the format of the income column.
- **isnull().sum() (after cleaning)**: This function is called again to verify that all missing values have been successfully removed. This ensures the cleaned datasets are now ready for further analysis.
- **display()**: This function is used to show the first few rows of both the cleaned training and test datasets. By doing this, it allows verification that the data has been correctly cleaned, standardized, and is in a suitable format for modeling.

The expected output shows the number of missing values in each column before and after cleaning. Initially, columns like `workclass`, `occupation`, and `native_country` have missing entries. After using `dropna()`, all missing values are removed, which is confirmed by a second check that shows zero missing values. The `income` column in the test dataset is also standardized by removing trailing periods. Finally, the first few rows of the cleaned datasets are displayed, confirming successful data cleaning and preparation, as shown:

Missing Values in the Traing Data Before Cleaning

```
age          0
workclass    1836
fnlwgt       0
education    0
education_num 0
marital_status 0
occupation   1843
relationship 0
race         0
sex          0
capital_gain 0
capital_loss 0
hours_per_week 0
native_country 583
income       0
dtype: int64
```

Missing Values in the Test Data Before Cleaning

```
age          0
workclass    963
fnlwgt       0
education    0
education_num 0
marital_status 0
occupation   966
relationship 0
race         0
sex          0
capital_gain 0
capital_loss 0
hours_per_week 0
native_country 274
income       0
dtype: int64
```

Missing Values in the Traing Data After Cleaning

```
age          0
workclass    0
fnlwgt       0
education    0
education_num 0
marital_status 0
occupation   0
relationship 0
race         0
sex          0
capital_gain 0
capital_loss 0
hours_per_week 0
native_country 0
income       0
dtype: int64
```

Missing Values in the Test Data After Cleaning

```
age          0
workclass    0
fnlwgt       0
education    0
education_num 0
marital_status 0
occupation   0
relationship 0
race         0
sex          0
capital_gain 0
capital_loss 0
hours_per_week 0
native_country 0
income       0
dtype: int64
```

Adult Training Data (after cleaning):

```
/var/folders/80/qns4qhrj56lg74phvxs3sj0r0000gn/T/ipykernel_58509/982283373.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
clean_adult_test['income'] = clean_adult_test['income'].str.replace('.', '')
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

Adult Test Data (after cleaning):

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
0	25	Private	226802	11th	7	Never-married	Machine-op-inspct	Own-child	Black	Male	0	0	40	United-States	<=50K
1	38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing	Husband	White	Male	0	0	50	United-States	<=50K
2	28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv	Husband	White	Male	0	0	40	United-States	>50K
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	Husband	Black	Male	7688	0	40	United-States	>50K
5	34	Private	198693	10th	6	Never-married	Other-service	Not-in-family	White	Male	0	0	30	United-States	<=50K

Encoding the categorical values

```
clean_adult_data['income'] = clean_adult_data['income'].apply(lambda x: 1 if x == '>50K' else 0)
clean_adult_test['income'] = clean_adult_test['income'].apply(lambda x: 1 if x == '>50K' else 0)

categorical_columns = ['workclass', 'education', 'marital_status', 'occupation',
                      'relationship', 'race', 'sex', 'native_country']

adult_data_encoded = pd.get_dummies(clean_adult_data, columns=categorical_columns)
adult_test_encoded = pd.get_dummies(clean_adult_test, columns=categorical_columns)

adult_test_encoded = adult_test_encoded.reindex(columns=adult_data_encoded.columns, fill_value=0)

print("\nAdult Training Data (after encoding):\n")
display(adult_data_encoded.head())
print("\nAdult Test Data (after encoding):\n")
display(adult_test_encoded.head())
```

The code snippet uses several functions to encode categorical variables into numerical values, preparing the data for machine learning models, which require numeric inputs.

- **apply(lambda x: ...):** This function converts the income column into a binary format where 1 represents '>50K' and 0 represents '<=50K'. This binary encoding allows the income variable to be used as the target in classification tasks.
- **pd.get_dummies():** One-hot encoding is performed on the specified categorical columns (workclass, education, marital_status, etc.) using this function. It transforms each categorical variable into a set of binary columns, making it possible for the model to process categorical data.
- **reindex():** To ensure the test data has the same set of columns as the training data after one-hot encoding, reindex() aligns the columns of the test DataFrame to match those in the training DataFrame, filling missing columns with zeros (fill_value=0).

- **display():** The first few rows of the encoded training and test datasets are displayed to confirm successful encoding and alignment.

The result is a fully numerical dataset where categorical variables are represented by binary columns, and the income column is converted into binary format.

Adult Training Data (after encoding):

```
/var/folders/80/qns4qhrj56lg74phvxs3sj0r0000gn/T/ipykernel_58509/934314687.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
clean_adult_data['income'] = clean_adult_data['income'].apply(lambda x: 1 if x == '>50K' else 0)
/var/folders/80/qns4qhrj56lg74phvxs3sj0r0000gn/T/ipykernel_58509/934314687.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
clean_adult_test['income'] = clean_adult_test['income'].apply(lambda x: 1 if x == '>50K' else 0)
```

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week	income	workclass_Federal-gov	workclass_Local-gov	workclass_Private	...	native_country_Portugal	native_c
0	39	77516	13	2174	0	40	0	False	False	False	...	False	
1	50	83311	13	0	0	13	0	False	False	False	...	False	
2	38	215646	9	0	0	40	0	False	False	True	...	False	
3	53	234721	7	0	0	40	0	False	False	True	...	False	
4	28	338409	13	0	0	40	0	False	False	True	...	False	

5 rows x 105 columns

Adult Test Data (after encoding):

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week	income	workclass_Federal-gov	workclass_Local-gov	workclass_Private	...	native_country_Portugal	native_c
0	25	226802	7	0	0	40	0	False	False	True	...	False	
1	38	89814	9	0	0	50	0	False	False	True	...	False	
2	28	336951	12	0	0	40	1	False	True	False	...	False	
3	44	160323	10	7688	0	40	1	False	False	True	...	False	
5	34	198693	6	0	0	30	0	False	False	True	...	False	

5 rows x 105 columns

4. Explore the data

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(14, 6))

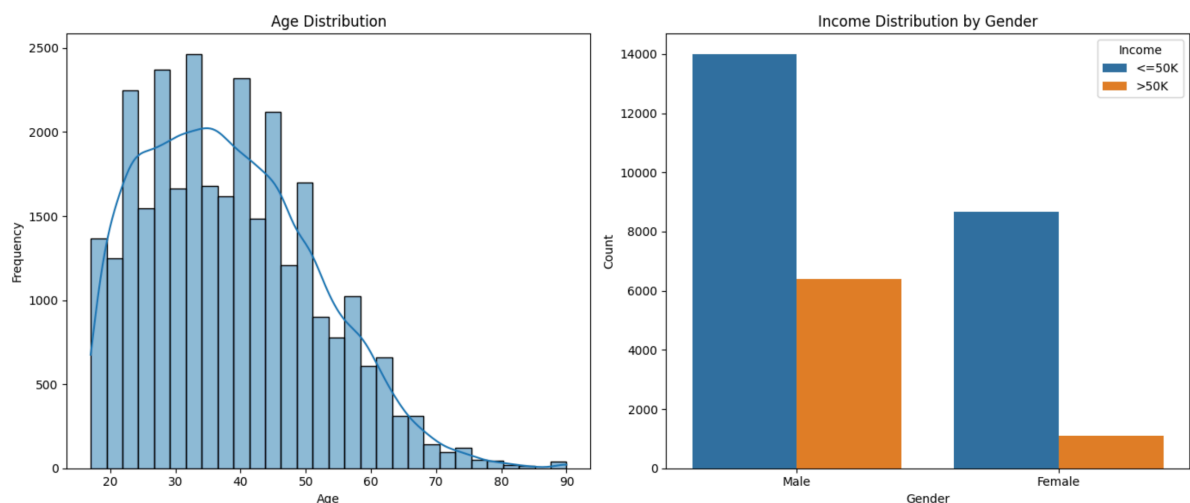
plt.subplot(1, 2, 1)
sns.histplot(clean_adult_data['age'], bins=30, kde=True)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
sns.countplot(data=clean_adult_data, x='sex', hue='income')
plt.title('Income Distribution by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(title='Income', labels=['<=50K', '>50K'])

plt.tight_layout()
plt.show()
```

The code uses visualization libraries to explore and understand the distribution of key features in the dataset. These plots help identify patterns and relationships between variables, providing insights that inform data preprocessing and model development.

- **import matplotlib.pyplot as plt and import seaborn as sns:** These libraries are imported for data visualization. matplotlib is used for creating general plots, while seaborn provides high-level functions for generating more complex and aesthetically pleasing visualizations.
- **plt.figure(figsize=(14, 6)):** This sets up the size of the entire plotting area, ensuring that both visualizations fit neatly side by side.
- **plt.subplot():** This function creates a subplot layout. The first subplot (1, 2, 1) displays the age distribution, while the second (1, 2, 2) shows the income distribution by gender.
- **sns.histplot():** Used to visualize the age distribution of individuals in the dataset, with 30 bins and a kernel density estimate (KDE) overlay. This plot provides insights into the age demographics of the population.
- **sns.countplot():** This function creates a count plot to compare income distribution across genders (male and female). The hue parameter breaks down the data by income, illustrating the proportion of individuals in each gender category earning $\leq 50K$ or $> 50K$.
- **plt.tight_layout() and plt.show():** `tight_layout()` adjusts the spacing between plots for a clean presentation, while `show()` renders the visualizations.



The data reveals key demographic trends related to age and income distribution. The age distribution shows that most individuals in the dataset are within the working-age range of 20 to 50 years, suggesting a focus on an economically active population. Additionally, the income distribution by gender highlights a significant disparity: a larger proportion of males fall into the ">50K" income category compared to females. This disparity suggests potential gender-based income inequalities, which could stem from differences in

occupation, education, or societal factors. Together, these insights underscore the importance of age and gender as factors in income determination.

5. Predictive Modeling

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report

pmodel = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    random_state=42
)

pmodel.fit(adult_data_encoded.drop('income', axis=1), adult_data_encoded['income'])

pred = pmodel.predict(adult_test_encoded.drop('income', axis=1))
```

The code uses the Gradient Boosting Classifier to train a predictive model for income classification. Below is a breakdown of the functions and parameters used in this code:

- **from sklearn.ensemble import GradientBoostingClassifier:** Imports the Gradient Boosting Classifier from Scikit-learn for building a predictive model using multiple decision trees.
- **from sklearn.metrics import accuracy_score, classification_report:** Imports functions for evaluating model performance—accuracy score and detailed classification metrics (precision, recall, F1-score).
- **GradientBoostingClassifier(...):** Creates a Gradient Boosting model with key parameters:
 - `n_estimators=100`: Uses 100 decision trees.
 - `learning_rate=0.1`: Controls tree contribution, balancing performance and computation.
 - `max_depth=5`: Limits tree depth to prevent overfitting.
 - `random_state=42`: Ensures reproducible results.
- **pmodel.fit(...):** Trains the model using features (`adult_data_encoded.drop('income', axis=1)`) and the target variable (`adult_data_encoded['income']`).
 - **pmodel.predict(...):** Predicts the income category (>50K or <=50K) for the test dataset (`adult_test_encoded.drop('income', axis=1)`).

Total no. of individuals who earn more than \$50k a year

```
totalnum = sum(pred == 1)
print(f"Number of individuals who earn more than USD 50,000/year: {totalnum}")

above50k = adult_test_encoded.index[pred == 1]
individuals = clean_adult_test.loc[above50k]

print("\nIndividuals who earn more than USD 50,000/year:\n")
display(individuals)
```

This code identifies the individuals in the test dataset who are predicted to earn more than USD 50,000 per year. Each function is does the following:

- **totalnum = sum(pred == 1):** Counts the total number of individuals predicted by the model to earn more than USD 50,000 (pred == 1).
- **print(f"...: {totalnum}"):** Prints the total count of individuals predicted to earn more than USD 50,000.
- **above50k = adult_test_encoded.index[pred == 1]:** Retrieves the indices of individuals in the test dataset where the income prediction is >50K.
- **individuals = clean_adult_test.loc[above50k]:** Extracts details of these high earners from the cleaned test dataset using their indices.
- **display(individuals):** Displays a table of individuals predicted to earn more than USD 50,000, showing their demographic details.

Number of individuals who earn more than USD 50,000/year: 3107

Individuals who earn more than USD 50,000/year:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	Husband	Black	Male	7688	0	40	United-States	1
7	63	Self-emp-not-inc	104626	Prof-school	15	Married-civ-spouse	Prof-specialty	Husband	White	Male	3103	0	32	United-States	1
10	65	Private	184454	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	6418	0	40	United-States	1
11	36	Federal-gov	212465	Bachelors	13	Married-civ-spouse	Adm-clerical	Husband	White	Male	0	0	40	United-States	0
14	48	Private	279724	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	3103	0	48	United-States	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
16254	38	Private	149347	Masters	14	Married-civ-spouse	Prof-specialty	Husband	White	Male	0	0	50	United-States	1
16255	43	Local-gov	23157	Masters	14	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	1902	50	United-States	1
16261	40	Private	202168	Prof-school	15	Married-civ-spouse	Prof-specialty	Husband	White	Male	15024	0	55	United-States	1
16278	38	Private	374983	Bachelors	13	Married-civ-spouse	Prof-specialty	Husband	White	Male	0	0	50	United-States	0
16280	35	Self-emp-inc	182148	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	60	United-States	1

:107 rows x 15 columns

ACCURACY

```
: accuracy = accuracy_score(adult_test_encoded['income'], pred)
print(f"Accuracy: {accuracy * 100:.3f}%")
```

Accuracy: 87.072%

- **accuracy_score(adult_test_encoded['income'], pred):** Calculates the accuracy by comparing true labels (adult_test_encoded['income']) with model predictions (pred).
- **Accuracy formula:** Ratio of correct predictions to total predictions.
- **print(f"Accuracy: {accuracy * 100:.3f}%"):** Prints accuracy as a percentage, formatted to three decimal places.

CONCLUSION

The study successfully utilized machine learning techniques to classify individuals into income categories based on demographic data. With an accuracy of over **87.07%**, the model proves to be a valuable tool for income prediction and socio-economic analysis. Further research could involve experimenting with different algorithms, incorporating additional features, or addressing class imbalance to improve predictive performance. The findings also highlight the potential for using demographic data in financial forecasting and economic policy development.

REFERENCE

Becker, B. & Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5XW20>